Level 1 OOP Challenges - Escape the Brainrot

res/challenges/level1_challenges.txt

ID: OOP-01

TYPE: PREDICT

DIFFICULTY: EASY

PROMPT:

What does this program print to standard output?

CODE:

```
class Person {
   String name = "Luigi";
}
public class Main {
   public static void main(String[] args) {
      Person p = new Person();
      System.out.println(p.name);
   }
}
```

ANSWER: Luigi

ID: OOP-02

TYPE: FILL_BLANK

DIFFICULTY: EASY

PROMPT:

Scenario: Instantiate a new Player object named 'hero' with 100 health.

Fill in the blank with the exact instantiation code to assign 'hero'.

CODE:

```
class Player {
   int hp;
   Player(int hp) { this.hp = hp; }
}
public class Main {
   public static void main(String[] args) {
      Player hero = ___;
   }
}
```

ANSWER: new Player(100)

ID: OOP-03

TYPE: PREDICT

DIFFICULTY: EASY

PROMPT:

What integer value is printed?

CODE:

```
class Counter {
   int count = 10;
   void increment() { count += 5; }
}
public class Main {
   public static void main(String[] args) {
      Counter c = new Counter();
      c.increment();
```

```
    System.out.println(c.count);
  }
}
```

ANSWER: 15

ID: OOP-04

TYPE: FILL_BLANK

DIFFICULTY: MEDIUM

PROMPT:

Scenario: In the Item constructor, assign the parameter 'name' to the instance field 'name' without renaming either variable.

Fill in the blank with the correct assignment statement (including semicolon).

CODE:

```
class Item {
  String name;

  Item(String name) {
    ___
  }
}
```

ANSWER: this.name = name;

ID: OOP-05

TYPE: PREDICT

DIFFICULTY: EASY

PROMPT:

Polymorphism and method overriding. What prints?

CODE:

```
class Animal {
  void speak() { System.out.println("Generic"); }
}
class Dog extends Animal {
  void speak() { System.out.println("Bark"); }
}
public class Main {
  public static void main(String[] args) {
    Animal a = new Dog();
    a.speak();
  }
}
```

ANSWER: Bark

ID: OOP-06

TYPE: FILL_BLANK

DIFFICULTY: EASY

PROMPT:

Scenario: You want 'Warrior' to inherit from 'Entity'. Which Java keyword belongs in the blank?

CODE:

```
class Entity {}

class Warrior ___ Entity {
```

```
    int armor = 10;
}
```

ANSWER: extends

ID: OOP-07

TYPE: PREDICT

DIFFICULTY: MEDIUM

PROMPT:

Using super to invoke the parent class method. What prints?

CODE:

```
class Base {
    void show() { System.out.print("A"); }
}
class Sub extends Base {
    void show() {
        super.show();
        System.out.print("B");
    }
}
public class Main {
    public static void main(String[] args) {
        new Sub().show();
    }
}
```

ANSWER: AB

ID: OOP-08

TYPE: FILL_BLANK

DIFFICULTY: MEDIUM

PROMPT:

Scenario: The Mage class inherits from Character. Inside the Mage constructor, invoke the parent constructor passing 'baseMana'.

Fill in the blank with the super constructor call (including semicolon).

CODE:

```
class Character {
   Character(int mana) {}
}


class Mage extends Character {
   Mage(int baseMana) {
      ___
   }
}
```

ANSWER: super(baseMana);

ID: OOP-09

TYPE: PREDICT

DIFFICULTY: MEDIUM

PROMPT:

Static variable modification across multiple instances. What prints?

CODE:

```
class Score {
```

```
  static int pts = 0;
  Score() { pts += 2; }
}
public class Main {
  public static void main(String[] args) {
    new Score();
    new Score();
    System.out.println(Score.pts);
  }
}
```

ANSWER: 4

ID: OOP-10

TYPE: FILL_BLANK

DIFFICULTY: HARD

PROMPT:

Scenario: Write the entire 'Key' class from scratch so that the Main program compiles and prints 101.

The class must have a public integer field named 'code' initialized by a single constructor accepting an integer parameter.

Write the complete class definition on one line (without extra spaces, e.g. class Key{public int code;Key(int c){code=c;}} or with this.code=code;).

CODE:

```
// Write the full class definition from scratch:
___


public class Main {
```

```
    public static void main(String[] args) {
        Key k = new Key(101);
        System.out.println(k.code);
    }
}
```

ANSWER: class Key{public int code;public Key(int code){this.code=code;}}